Project Plan

Team contract, requirements, approach, tasks and hours, meetings, strengths and weaknesses

Team	Distributed Developments (group project)
Members	Mohammed Nurul Hasan (S376146)
	Md Rakibul Hassan Emon (S375332)
	Md Modasser Hossain (S375173)
Product	Library Platform, .NET 10 ASP.NET Core MVC
Version	1.0
Date	16 October 2026

Contents

1. Project summary . . . 3
2. Team and responsibilities . . . 3
3. Team contract . . . 3
4. Requirements notes . . . 4
5. Approach . . . 5
6. Project management site and marking access . . . 6
7. Billable tasks and hours . . . 6
8. Meetings and minutes . . . 7
9. Strengths and weaknesses . . . 8

Appendix A. Task list with estimates . . . 11

Appendix B. Weekly hours log . . . 13

Appendix C. Meeting minutes . . . 14

1. Project summary

Project	Library Platform: expansion of the Assignment 2 library system into a multi-channel, multi-branch platform ("Distributed Development")
Client request	One coherent .NET 10 ASP.NET Core MVC application with seven new features: self-service kiosk, public read-only REST API, simulated notifications, multi-branch inventory and transfers, reservations and waitlists, data importer, manager analytics with CSV and PDF export
Baseline	The team's own Assignment 2 system (items, borrowing, returns, late fines, borrowers, admin / reception / manager roles, public search)
Team	Three members (section 2)
Repository	<https://github.com/nurulhasan2k/library-platform.git> (branch `main`)
Project board	GitHub Projects board of the same repository: 68 cards, one per task, each with an owner, milestone and estimate
Submission	16 October 2026, 23:59
Status at the date of this plan	All seven features and the baseline are built, integrated and tested. Remaining: test on the assessor's virtual machine image and final read-through.

2. Team and responsibilities

	Mohammed Nurul Hasan	Md Rakibul Hassan Emon	Md Modasser Hossain
Student ID	S376146	S375332	S375173
Theme	Foundation, branches and access	Lending and patron experience	Catalogue services, import and reporting
Client features	3.4 Multi-branch inventory and transfers; shared foundation (MVC / .NET 10, data, roles, seed and reset)	3.1 Self-service kiosk; 3.3 Notifications; 3.5 Reservations and waitlists	3.2 Public REST API; 3.6 Data importer; 3.7 Manager analytics and export
Integration chains led	7 (and the branch-aware half of 1, 3, 4, 6)	1, 2, 3	4, 5, 6
Documents	Installation and Running Instructions; README assembly; repository hygiene	User Guide; Project Plan	Group Portfolio PDF
Code review	Reviews Rakibul's work	Reviews Modasser's work	Reviews Mohammed's work

Every client requirement has exactly one owner (the traceability matrix is in the Group Portfolio, section 6).

3. Team contract

Draft for all three members to read and confirm. Each member signs (or types their name) in the table at the end.

Purpose. We want one working, well-documented product that every member understands and can demonstrate, delivered on time and with the work shared fairly.

Communication.

- One team chat channel for day-to-day questions. Written check-in (what I did, what I will do, what blocks me) at least twice a week.
- Reply to a direct question within 24 hours (48 hours at weekends). If someone will be away for more than two days they say so in advance.

- Anything that changes the shared contract (service interfaces, data model, settings) is announced in the chat before it is merged and recorded in `docs/DECISIONS.md`.

Working rules. These are the rules in `CONTRIBUTING.md`:

- `main` always builds and runs; nobody commits straight to it.
- Work happens on short-lived Git branches named `feature/<area>-<short-name>` (and `fix/` or `docs/`). A "branch" in the application is always a library branch; a Git branch is always called a Git branch.
- Every change is a pull request using the template and is reviewed by one teammate (Mohammed reviews Rakibul, Rakibul reviews Modasser, Modasser reviews Mohammed) within one working day.
- Done means: merged, works with the seed data, validated and authorised, has tests for its rules, has its README section and user-guide section written.
- Dates in code come from `IClock`; secrets, data files and build output are never committed.

Decisions. We aim for agreement. If we cannot agree after one discussion, the person who owns the area decides, writes the reason in `docs/DECISIONS.md` and the others may challenge it once, in writing.

Workload and fairness. Estimates are in section 7. Everyone logs their own hours weekly. If someone is more than a week behind or well ahead, the others rebalance tasks rather than silently absorbing or dropping work. Each person is accountable for the quality of the tasks they own.

Conflict. Raise it in the chat first, then in a call. If it is still unresolved, ask the unit tutor for advice.

Integrity. Each person's contributions are recorded truthfully in the hours log and the commit history. Any use of AI or other assistance, and any reused or third-party material, is declared in line with the unit's rules.

Member	Agreed (name and date)
Mohammed Nurul Hasan (S376146)	Agreed, by message to the team, 16 Oct 2026
Md Rakibul Hassan Emon (S375332)	Agreed, by message to the team, 16 Oct 2026
Md Modasser Hossain (S375173)	Agreed, by message to the team, 16 Oct 2026

4. Requirements notes

4.1 What the client asked for

#	Feature	What the client expects to see
3.1	Self-service kiosk	Patron signs in with a card number and PIN, sees an account summary (loans, due dates, fines, holds), checks out items by library code, gets a receipt, is signed out after inactivity; simple large-button screens
3.2	Public read-only REST API	Items, categories, branches, status; read-only, protected by a key, rate limited, documented and easy to test
3.3	Notifications	Borrowed, due soon, fine accruing, reservation ready, reservation expired; email and SMS simulated on a dashboard or console; patron preference
3.4	Multi-branch inventory	Every item belongs to a branch; staff choose their branch; search by branch; transfers between branches with in-transit handling; opening hours
3.5	Reservations and waitlists	Join a queue for an unavailable item; next person is identified and notified when it returns; the item is held for them
3.6	Data importer	CSV (and an external provider) with validation, preview, import into a branch and an import history; helpful errors

#	Feature	What the client expects to see
3.7	Manager analytics	Borrowing, fine revenue and inventory-health reports with charts and tables; export to CSV and PDF
4	Integration	Seven workflows that cross features must work end to end on the same data
6	Seed and reset	A repeatable multi-branch demo data set and a reset procedure
9	Demonstration	Twelve demo items that must work on a fresh install on the virtual machine
10	Deliverables	Repository with README (14 topics), Installation and Running Instructions, User Guide, Project Plan, Group Portfolio PDF

4.2 Assumptions and questions

Topic	Our assumption	Status
Meaning of "branch" in the install instructions	A physical library branch inside the application (not a Git branch)	Assumed; to be confirmed if the client replies
Baseline	The team's Assignment 2 system	Confirmed within the team
Reservations	Per physical copy (one library code), not per title	Assumed; listed as a limitation
Notifications	Simulated: stored, shown on a page and logged to the console	Assumed acceptable ("dashboard or console")
API security	An API key in a header plus rate limiting is enough; one shared demo key	Assumed
Barcode scanners	Typing (or a scanner that types and presses Enter) is acceptable for the kiosk	Assumed
Virtual machine	Has, or can install, the .NET 10 SDK; the project needs no other download	Installation guide covers it
Database	Not specified. A JSON data file behind a storage interface is used so nothing needs installing	Decision D1

4.3 Constraints

.NET 10 ASP.NET Core MVC; installs and runs from the Git repository by following the README; no placeholder screens; admin tools are never public; all input (including imported CSV) is validated with useful messages; understandable, maintainable and demonstrable.

5. Approach

Contract first, then parallel work. Before anyone built features we fixed the shared contract: the service interfaces (`IClock` , `IBranchContext` , `IPatronAuthService` , `ILendingService` , `ITransferService` , `ICatalogueQuery` , `INotificationService` , `IReservationService`), the data model, the roles matrix and the seed scenarios. Each area could then be built and tested against the same contract, and stubs let the solution build from day one. The stubs were later replaced by the real services.

One codebase, many front doors. Business rules live in `Library.Core` services; the web project only maps requests to services. The kiosk, the desk screens and the web account all call the same `ILendingService` , so a loan made anywhere updates the same data, triggers the same hold and notification hooks, and shows up in the same reports.

The hook rule. Lending asks reservations whether the item may be lent, then records a notification; a return offers the item to the next person waiting. Hooks never make a loan or return fail.

Phases. 0 Kickoff (repository, standards, contract, ER draft, seed scenarios, project board); 1 Foundation (port to MVC / .NET 10, data, roles, branches, core seed); 2 Features (seven features in parallel against the contract); 3 Integration (the seven chains and the twelve demo items end to end); 4 Documentation (README, installation, user guide, plan, portfolio).

Quality. Automated tests for every rule (117 tests), a scripted browser run of the twelve demo items (67 checks), pull-request review, a seed and reset command so every test starts from known data, and a manual fresh-clone check.

Technology choices and why are in `docs/DECISIONS.md`. The main ones: cookie authentication with three schemes (staff, patron, kiosk); PBKDF2 for passwords and PINs; a JSON file behind `ILibraryStore`; hand-written CSV and PDF writers and OpenAPI document so that nothing has to be downloaded to build.

6. Project management site and marking access

Item	Where
Source repository	<https://github.com/nurulhasan2k/library-platform.git>, branch `main`
Project board	The repository's **Projects** tab (GitHub Projects): 68 cards, each with an owner, a milestone (M1, M2, M3) and an hours estimate
Documentation	`README.md`, and the `docs/` folder (installation, user guide, architecture, decisions, data model, seed data, configuration, demo walkthrough)
Marker access	The repository is public, so no invitation is needed (made public 16 October 2026)
Chat	Facebook Messenger (group chat)

7. Billable tasks and hours

Estimates come from the project board (68 cards). The *Hours logged* column shows the hours each member reports having actually spent (rough totals given by the members themselves, not timesheets); a blank means not yet provided; the full task list is in Appendix A.

Team member	Milestone 1 (kickoff)	Milestone 2 (midpoint)	Milestone 3 (final)	Estimated total	Hours logged
Mohammed Nurul Hasan	7	78	40	125	about 20 (self-reported)
Md Rakibul Hassan Emon	6	59	64	129	about 13 (self-reported)
Md Modasser Hossain	4	63	70	137	about 16 (self-reported)
Team				**391**	about 49 (self-reported)

7.1 Contribution record

The work was **planned** as an equal three-way split: the board estimates are 125, 129 and 137 hours, which is 32%, 33% and 35% of the 391-hour total, and each member owned a third of the client features. What each member actually contributed is recorded below from the Git history (all branches, to 16 October 2026). Commit counts are only one measure; hours and the work itself are the others.

Team member	Commits	Work recorded in the repository	Hours logged
Mohammed Nurul Hasan (S376146)	3	Initial solution skeleton, shared service contracts and kickoff documents; project board and team process; the integrated build of the whole platform with its automated tests, scripts, sample files, documentation and the submission documents. Self-reported: solution structure and project skeleton; shared service contracts; initial documentation and kickoff materials; repository link in the README; integrating all seven features into the final application; checking that the components work together; reviewing and consolidating the tests and documentation. Work ran from 7 to 16 October 2026	about 20 (self-reported)
Md Rakibul Hassan Emon (S375332)	1	Kiosk, notifications and reservations (pull request #70). Self-reported: the kiosk, the notification features and the reservation functionality; fitting them into the application structure; checking the functionality and fixing integration issues. Work was mostly on the kiosk, notifications and reservations	about 13 (self-reported)
Md Modasser Hossain (S375173)	11	Public API: pipeline with key check, rate limit, CORS and JSON errors; items, categories, branches and status endpoints; error handling; endpoint and pipeline tests. Importer: CSV validation, sample files, admin import screens with preview, confirm, history and error download, simulated provider. Manager reports with calculators, exports and screens. Self-reported: most of the work in the week of 12 to 16 October 2026, including debugging, integration and verification	about 16 (self-reported)

7.2 Use of AI assistance

The integrated build that is submitted (the combined application, its automated tests, the scripts, sample import files, the documentation and these submission documents) was produced with the help of Claude, an AI assistant made by Anthropic, working from the team's Assignment 2 baseline, shared contracts and feature plan. Mohammed Nurul Hasan directed that work, ran the result and committed it. The AI assistant also helped draft the planning and documentation text. Rakibul and Modasser each wrote their own feature branches earlier; Mohammed reports that neither of them used any AI tool for their feature branches. Anyone marking this work should treat the integrated build as AI-assisted.

How the pieces fit. Rakibul's and Modasser's feature work was built in parallel against the shared contracts and the original plan, and is kept in the repository history. The version submitted is the integrated build committed by Mohammed, which brings all seven features and the baseline into one application, so the submitted code is not the same code as the earlier feature branches. Hours are rough self-reported totals; individual reflections are in section 9.3.

Delivered differently from the first plan: card A3 ("EF Core + SQLite") was delivered as a JSON store behind `ILibraryStore`, and cards C5, C12 (CSV/PDF libraries, Swagger) were delivered with hand-written code. The reasons and the trade-off are decisions D1, D11 and D12 in `docs/DECISIONS.md`.

8. Meetings and minutes

The plan was for three meetings: Kickoff, Midpoint and Final, each with an agenda, a minimum result, a chair and a note-taker.

Meeting	Planned purpose	Status	Where its decisions are recorded
1 Kickoff	Agree the plan and freeze the contract before anyone builds	**Not held as a meeting** (time constraints)	The 14 decisions and 2 contract additions it was to confirm are in `docs/DECISIONS.md`, `docs/ARCHITECTURE.md` and `docs/SEED-SCENARIOS.md`
2 Midpoint	Check the foundation, see each main path working, decide what to cut	**Not held as a meeting**	Progress is visible on the project board and in the test results
3 Final	Run the whole demo list on the virtual machine, close every deliverable	**Not held as a meeting**	The demo run-through is documented in `docs/DEMO-WALKTHROUGH.md` and the Group Portfolio (section 7)

No minutes exist for meetings that did not take place, and none have been written after the fact. The planned agendas, the scripts and a blank minutes template for each meeting are kept with the team's working files. If the team holds a short final review before submission, its real minutes (date, attendees, decisions, actions) should be added here as Appendix B.

9. Strengths and weaknesses

9.1 The product

Strengths

- The seven features share one set of data and one set of rules, so the integration chains genuinely work end to end (for example a kiosk loan creates a notification and closes the borrower's own hold; a return offers the item to the next person).
- Every rule has an automated test, and the twelve demo items are checked by a scripted browser run.
- It installs with only the .NET 10 SDK: no packages to download, no database server, and a one-command reset to known demo data.
- Security basics are covered: hashed passwords and PINs, PIN lock-out, anti-forgery tokens, role checks on every staff area, a read-only key-protected rate-limited API, formula-safe CSV export.
- The public pages work on phones, and the kiosk has its own simple large-button layout.

Weaknesses and limitations

- A JSON data file is simple and portable but is not meant for a very large library or many simultaneous writers.
- Email and SMS are simulated, there is one shared demo API key, and the site runs on plain HTTP for the demo.
- Holds are per physical copy, not per title.
- CSV and PDF output come from small hand-written writers: reliable for these reports but with a plainer look than a commercial library would give.
- It has been tested in a clean Linux environment with the .NET 10 SDK and in a scripted browser, but not yet on the assessor's own virtual machine image.

9.2 The team process

Strengths: the contract-first approach and the shared review rules kept three people's work compatible; the project board gave every task an owner and an estimate; the seed scenarios were agreed before the features were built so every demo has data.

Weaknesses: the three planned meetings were not held, so decisions have no minutes and were recorded only in the decision log; hours were estimated rather than logged as the work happened, and the hours reported at the end are much lower than the 391-hour board estimate, so the estimates were not a reliable guide to the real effort.

9.3 Individual reflections

Each member's own words, lightly shortened.

Member	Reflection
Mohammed Nurul Hasan (S376146)	What went well: dividing the project into separate features, developing them collaboratively, then bringing everything together. Challenge: integrating the components into one working application in the time available, especially keeping the features built by different members consistent. Next time: set clearer integration milestones and do integration testing earlier so compatibility problems show up before the final stage.
Md Rakibul Hassan Emon (S375332)	What went well: implementing several user-facing features that support the library's day-to-day operations. Challenge: making the kiosk, notifications and reservations work consistently with the rest of the application. Next time: allow more time for integration testing and coordinate feature development earlier so the final integration is smoother.
Md Modasser Hossain (S375173)	What went well: building several backend features and connecting them with the platform's administrative and reporting functions. Challenge: keeping API behaviour, validation and error handling consistent across components. Next time: set up a more structured testing and integration process earlier so problems show up sooner.

Appendix A. Task list with estimates

Id	Task	Owner	Milestone	Est. hours	Hours logged
A1	Repo and standards	Mohammed Nurul Hasan	M1	7	
A2	Port to .NET 10 MVC	Mohammed Nurul Hasan	M2	14	
A3	EF Core + SQLite	Mohammed Nurul Hasan	M2	8	
A4	Auth and roles	Mohammed Nurul Hasan	M2	8	
A5	Shared layout, theme and error handling	Mohammed Nurul Hasan	M2	6	
A6	Branch model and admin	Mohammed Nurul Hasan	M2	8	
A7	Branch selection and context	Mohammed Nurul Hasan	M2	5	
A8	Branch-aware lending (ILendingService)	Mohammed Nurul Hasan	M2	10	
A9	Fines paid tracking	Mohammed Nurul Hasan	M2	3	
A10a	Transfer service and rules (3.4)	Mohammed Nurul Hasan	M2	6	
A10b	Transfer screens and lists	Mohammed Nurul Hasan	M3	6	
A11	Public site upgrade	Mohammed Nurul Hasan	M3	6	
A12a	Seed skeleton and core seed	Mohammed Nurul Hasan	M2	4	
A12b	Rich seed, reset and seeding doc	Mohammed Nurul Hasan	M3	4	
A13a	Tests, foundation	Mohammed Nurul Hasan	M2	3	
A13b	Tests, transfers and integration	Mohammed Nurul Hasan	M3	3	
A14	README assembly and Installation guide	Mohammed Nurul Hasan	M3	8	
A15	Integration and fresh-install test on the VM	Mohammed Nurul Hasan	M3	6	
A16	Portfolio and plan inputs	Mohammed Nurul Hasan	M3	4	
R-A-M2	Reviews and support	Mohammed Nurul Hasan	M2	3	
R-A-M3	Reviews and support	Mohammed Nurul Hasan	M3	3	
B1	Wireframes and rules	Md Rakibul Hassan Emon	M1	4	
B2	Kiosk shell and station setup	Md Rakibul Hassan Emon	M2	6	
B3	Kiosk sign-in and session	Md Rakibul Hassan Emon	M2	6	
B4	Kiosk checkout	Md Rakibul Hassan Emon	M2	10	
B5	Kiosk account summary	Md Rakibul Hassan Emon	M3	6	
B6	Notification service and templates	Md Rakibul Hassan Emon	M2	8	
B7	Notification dashboard	Md Rakibul Hassan Emon	M2	6	
B8	Scheduler and simulated date	Md Rakibul Hassan Emon	M3	8	
B9	Reservation rules and service	Md Rakibul Hassan Emon	M2	12	
B10	Reservation screens	Md Rakibul Hassan Emon	M3	8	
B11	Integration with A's lending	Md Rakibul Hassan Emon	M3	5	
B12	Seeders	Md Rakibul Hassan Emon	M2	4	
B13a	Tests, rules and services	Md Rakibul Hassan Emon	M2	4	

Id	Task	Owner	Milestone	Est. hours	Hours logged
B13b	Tests, scheduler and kiosk	Md Rakibul Hassan Emon	M3	4	
B14	User Guide (patron-facing)	Md Rakibul Hassan Emon	M3	10	
B15	README sections	Md Rakibul Hassan Emon	M3	4	
B16	Journeys and portfolio inputs	Md Rakibul Hassan Emon	M3	4	
B17	Integration test of chains 1 to 3	Md Rakibul Hassan Emon	M3	6	
B18a	Project Plan set-up	Md Rakibul Hassan Emon	M1	2	
B18b	Project Plan compilation	Md Rakibul Hassan Emon	M3	6	
R-B-M2	Reviews and support	Md Rakibul Hassan Emon	M2	3	
R-B-M3	Reviews and support	Md Rakibul Hassan Emon	M3	3	
C1	Library spike and API design	Md Modasser Hossain	M1	4	
C2	API: catalogue endpoint	Md Modasser Hossain	M2	8	
C3	API: categories and status	Md Modasser Hossain	M2	6	
C4	API security and limits	Md Modasser Hossain	M2	7	
C5	API documentation and examples	Md Modasser Hossain	M3	5	
C6	CSV import: parsing and validation	Md Modasser Hossain	M2	12	
C7	Import preview and commit	Md Modasser Hossain	M3	10	
C8	Mock external provider	Md Modasser Hossain	M3	6	
C9	Analytics: borrowing statistics	Md Modasser Hossain	M2	8	
C10	Analytics: fine revenue audit	Md Modasser Hossain	M2	8	
C11	Analytics: inventory health	Md Modasser Hossain	M3	8	
C12	Exports (CSV and PDF)	Md Modasser Hossain	M3	12	
C13	Manager dashboard and navigation	Md Modasser Hossain	M3	4	
C14	Seeders and sample files	Md Modasser Hossain	M2	3	
C15a	Tests, API and validation	Md Modasser Hossain	M2	5	
C15b	Tests, reports and exports	Md Modasser Hossain	M3	5	
C16	README sections	Md Modasser Hossain	M3	4	
C17	Integration test of chains 4 to 6	Md Modasser Hossain	M3	6	
C18a	Portfolio set-up	Md Modasser Hossain	M2	4	
C18b	Group Portfolio PDF compilation	Md Modasser Hossain	M3	8	
R-C-M2	Reviews and support	Md Modasser Hossain	M2	2	
R-C-M3	Reviews and support	Md Modasser Hossain	M3	2	

Appendix B. Weekly hours log

Week starting	Mohammed Nurul Hasan (hours / what)	Md Rakibul Hassan Emon (hours / what)	Md Modasser Hossain (hours / what)

Appendix C. Meeting minutes

Add the real minutes of any meeting the team actually holds here (date and time, attendees, decisions, actions with owners).